

TASK 2 A & B

1. Develop a C program that uses the system calls `open()`, `read()`, `write()`, and `close()` to copy the contents of one file to another. Explain how control transitions between user space and kernel space during execution.

Use the `strace` utility to trace all system calls generated by the command `cat sample.txt`. Analyze the sequence of system calls and identify the kernel services involved.

open()

```
int src = open(argv[1], O_RDONLY);  
int dst = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0644);
```

read()

```
while ((n = read(src, buffer, BUF_SIZE)) > 0)
```

write()

```
write(dst, buffer, n);
```

close()

```
close(src);  
close(dst);
```

OUTPUT:

```

gopal@PC: /tmp
gopal@PC:~$ su gopal
Password:
gopal@PC:/root$ cd /tmp
Command 'cmd' not found, but there are 22 similar ones.
gopal@PC:/root$ cd /tmp
gopal@PC:/tmp$ rm -f task2.c task2
gopal@PC:/tmp$ printf "#include <stdio.h>\n#include <stdlib.h>\n#include <fcntl.h>\n#
include <unistd.h>\n#define BUF_SIZE 1024\nint main(int argc, char *argv[]) {\n
    if (argc != 3) {\n        printf("Usage: %s <source> <destination>\n", argv[0]);\n
        exit(1);\n    }\n    int src = open(argv[1], O_RDONLY);\n    if (src <
0) { perror("open source"); exit(1); }\n    int dst = open(argv[2], O_WRONLY |
O_CREAT | O_TRUNC, 0644);\n    if (dst < 0) { perror("open destination"); exit(1);
}\n    char buffer[BUF_SIZE];\n    ssize_t n;\n    while ((n = read(src, buffer,
BUF_SIZE)) > 0) {\n        write(dst, buffer, n);\n    }\n    close(src);\n
    close(dst);\n    printf("File copied successfully.\n");\n    return 0;\n}\n
gopal@PC:/tmp$ cat task2.c
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#define BUF_SIZE 1024

int main(int argc, char *argv[]) {
    if (argc != 3) {
        printf("Usage: %s <source> <destination>\n", argv[0]);
        exit(1);
    }

    int src = open(argv[1], O_RDONLY);
    if (src < 0) { perror("open source"); exit(1); }

    int dst = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (dst < 0) { perror("open destination"); exit(1); }

    char buffer[BUF_SIZE];
    ssize_t n;
    while ((n = read(src, buffer, BUF_SIZE)) > 0) {
        write(dst, buffer, n);
    }
    close(src);
    close(dst);
    printf("File copied successfully.\n");
    return 0;
}
gopal@PC:/tmp$

```

gopal@PC: /tmp

```
gopal@PC:/tmp$ cat task2.c
```

```
#define BUF_SIZE 1024
```

```
int main(int argc, char *argv[]) {  
    if (argc != 3) {  
        printf("Usage: %s <source> <destination>\n", argv[0]);  
        exit(1);  
    }  
  
    int src = open(argv[1], O_RDONLY);  
    if (src < 0) { perror("open source"); exit(1); }  
  
    int dst = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0644);  
    if (dst < 0) { perror("open destination"); exit(1); }  
  
    char buffer[BUF_SIZE];  
    ssize_t n;  
    while ((n = read(src, buffer, BUF_SIZE)) > 0) {  
        write(dst, buffer, n);  
    }  
  
    close(src);  
    close(dst);  
    printf("File copied successfully.\n");  
    return 0;  
}
```

```
gopal@PC:/tmp$ gcc task2.c -o task2
```

```
gopal@PC:/tmp$ ./task2.c
```

```
bash: ./task2.c: Permission denied
```

```
gopal@PC:/tmp$ gcc task2.c -o task2
```

```
gopal@PC:/tmp$ ./task2
```

```
Usage: ./task2 <source> <destination>
```

```
gopal@PC:/tmp$ echo "This is a test file for task 2" > source.txt
```

```
gopal@PC:/tmp$ ./task2 source.txt dest.txt
```

```
File copied successfully.
```

```
gopal@PC:/tmp$ cat dest.txt
```

```
This is a test file for task 2
```

```
gopal@PC:/tmp$
```